

Serverless Web Application with AWS Lambda and API Gateway

Services: Lambda, API Gateway, S3, DynamoDB, Cognito

Goal: Develop a fully serverless web application where authenticated users can securely upload files. Files are stored in Amazon S3, metadata is maintained in DynamoDB, and API requests are processed through API Gateway and Lambda functions.

Skills: API integration, serverless architecture, authentication.

Definition

A Serverless Web Application is a cloud-native application architecture where infrastructure management is handled by AWS services, allowing developers to focus only on application logic.

In this project, users authenticate using Amazon Cognito and upload files through a web interface. API Gateway receives client requests and invokes AWS Lambda functions. Uploaded files are stored in Amazon S3 while file metadata is stored in Amazon DynamoDB.

The architecture eliminates the need for server management and provides automatic scalability, high availability, and cost optimization.

Scope of the Project

The scope of this project includes designing and implementing a secure serverless file management system using AWS services.

The project focuses on:

- User authentication using Amazon Cognito
- Secure API development using API Gateway
- Serverless backend processing using AWS Lambda
- File storage using Amazon S3
- Metadata storage using DynamoDB
- Access control using IAM Roles and Policies
- Static frontend hosting
- Event-driven cloud architecture

This solution represents a modern cloud-native application suitable for production environments.

Methodologies

The project follows a Serverless Architecture Model consisting of four major layers.

1. Presentation Layer

Provides the user interface through a web application hosted on Amazon S3.

Responsibilities:

- User registration
 - User login
 - File selection
 - File upload
-

2. Authentication Layer

Implemented using Amazon Cognito.

Responsibilities:

- User registration
 - User authentication
 - Token generation
 - Session management
-

3. Application Layer

Implemented using Amazon API Gateway and AWS Lambda.

Responsibilities:

- Receive upload requests
 - Process file uploads
 - Validate requests
 - Store file information
-

4. Data Layer

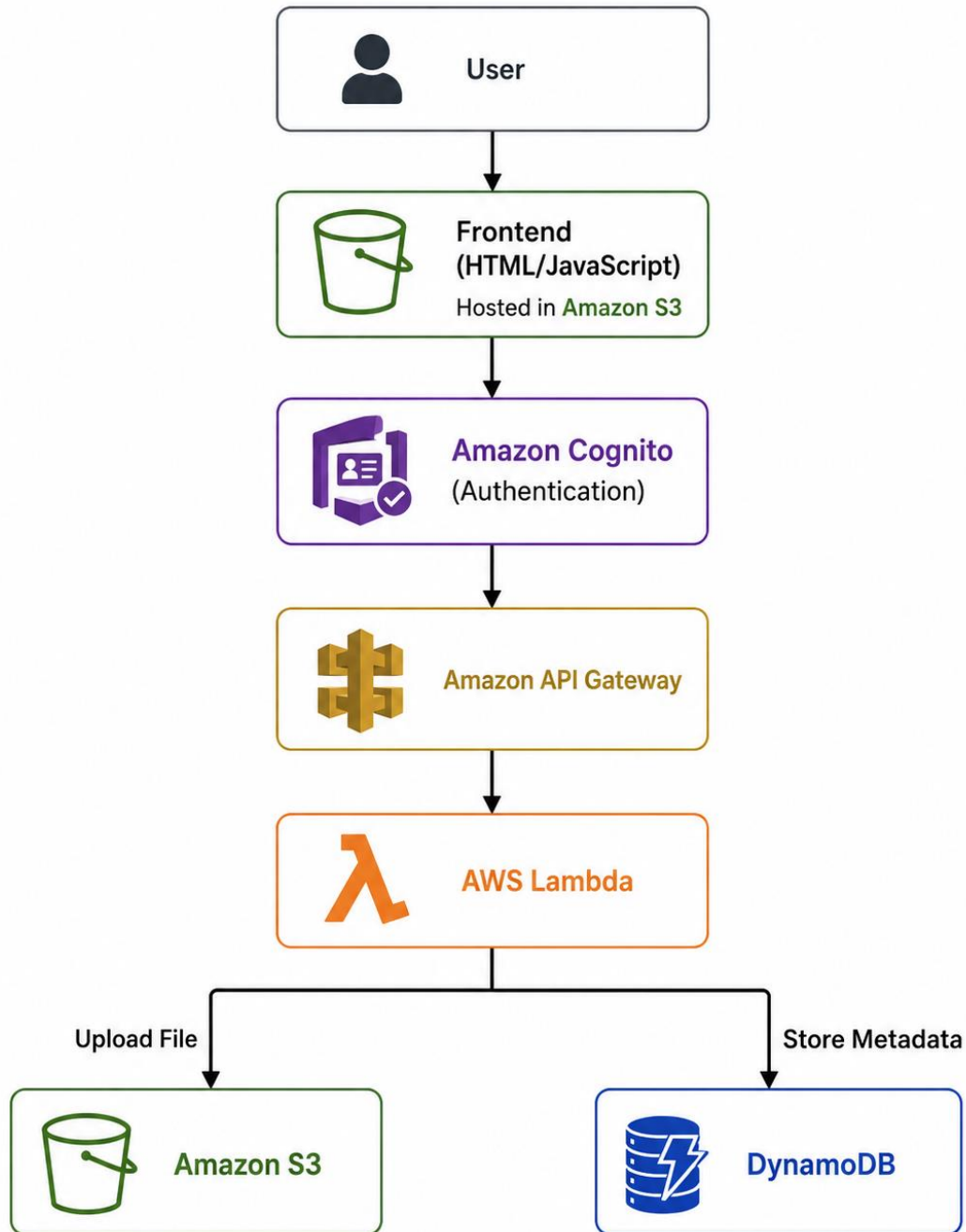
Implemented using Amazon S3 and DynamoDB.

Responsibilities:

- Store uploaded files

- Store metadata records
- Maintain file information

Architecture Diagram / Flow



Explanation of Services Used (Purpose)

Amazon Cognito:

Amazon Cognito provides authentication, authorization, and user management services.

Purpose:

- User registration
 - User login
 - JWT token generation
 - Secure access control
-

Amazon API Gateway:

API Gateway creates and manages REST APIs.

Purpose:

- Receive HTTP requests
 - Route requests to Lambda
 - Secure APIs using Cognito authorization
-

AWS Lambda:

AWS Lambda executes backend code without provisioning servers.

Purpose:

- Process upload requests
 - Store files in S3
 - Save metadata in DynamoDB
-

Amazon S3:

Amazon Simple Storage Service provides object storage.

Purpose:

- Store uploaded files
 - Provide scalable and durable storage
-

Amazon DynamoDB:

DynamoDB is a fully managed NoSQL database service.

Purpose:

- Store file metadata

- Maintain file records
- Support fast lookups

IAM Roles:

Identity and Access Management controls permissions between AWS services.

Purpose:

- Allow Lambda access to S3
- Allow Lambda access to DynamoDB
- Maintain security best practices

CloudWatch:

CloudWatch provides monitoring and logging.

Purpose:

- Monitor Lambda execution
- View application logs
- Troubleshoot errors

Step-by-Step Procedure

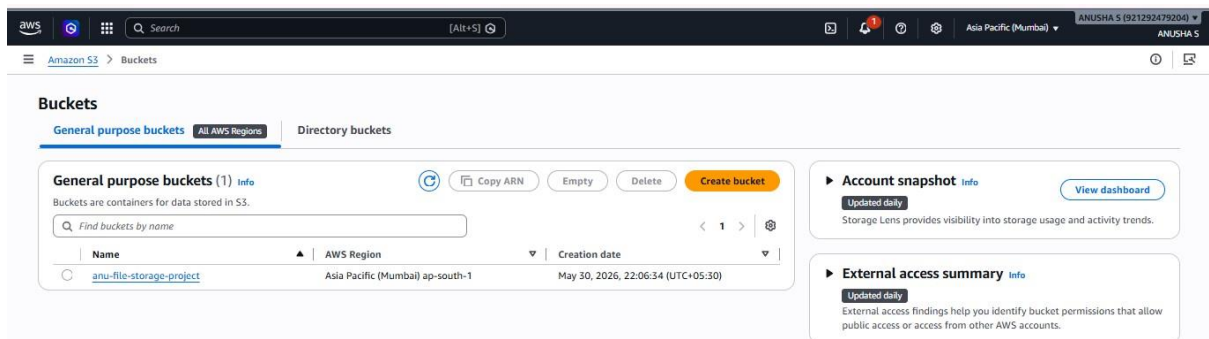
Step 1: Create S3 Bucket

Create an S3 bucket to store uploaded files.

Configuration:

Bucket Name: anu-file-storage-project

Region: ap-south-1



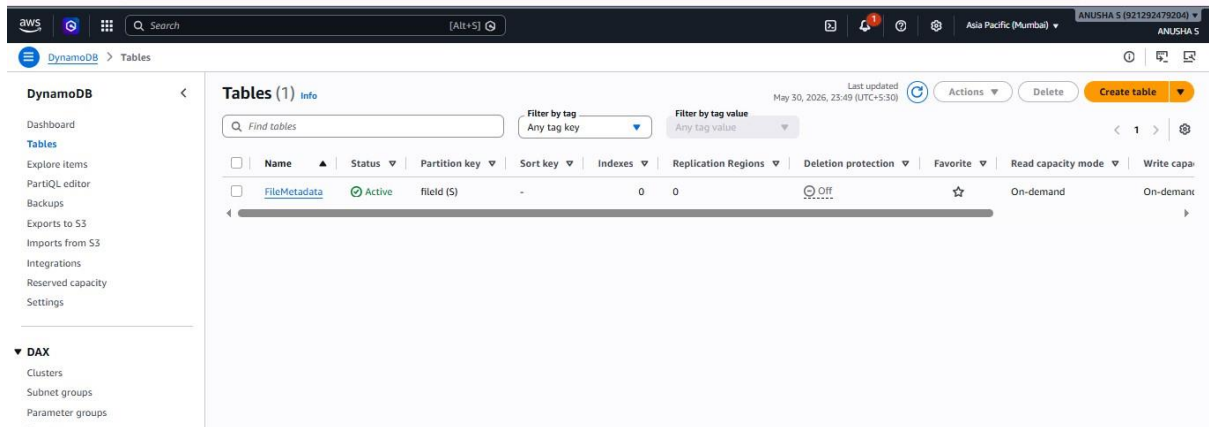
Step 2: Create DynamoDB Table

Create DynamoDB table.

Configuration:

Table Name: FileMetadata

Partition Key: fileId (String)



Step 3: Create Cognito User Pool

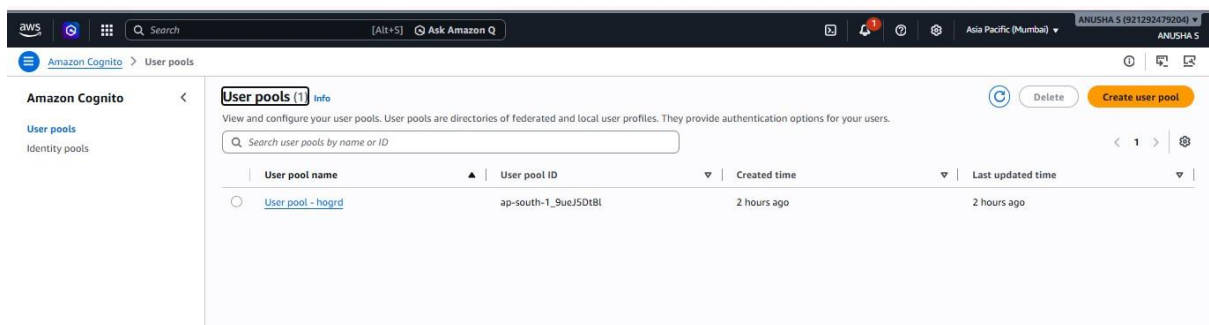
Configure Amazon Cognito.

Settings:

User Pool Name: User pool – hogrd

Sign-In Method: Email

Self Registration: Enabled



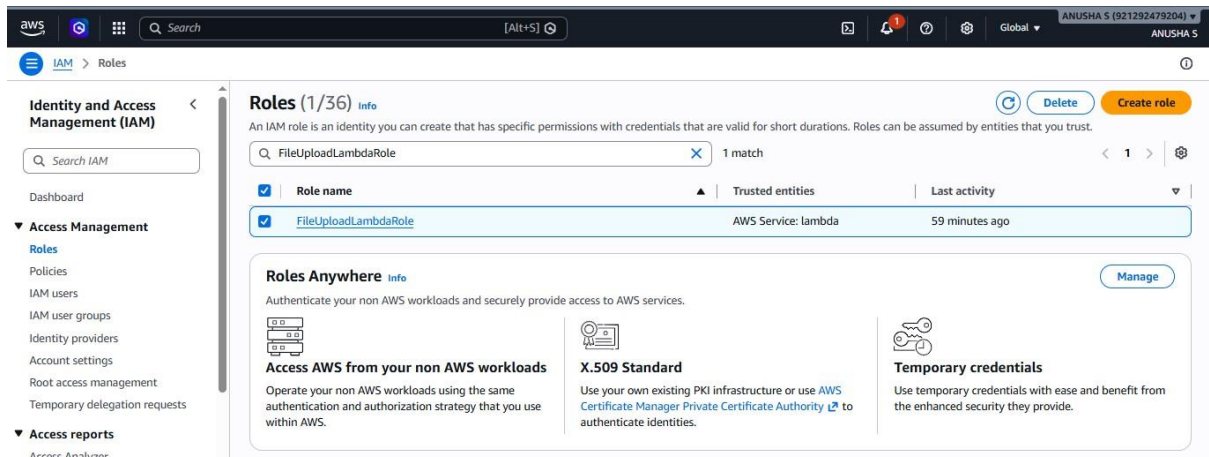
Step 4: Create IAM Role

Create Lambda execution role.

Policies Attached: AWSLambdaBasicExecutionRole

AmazonS3FullAccess

AmazonDynamoDBFullAccess

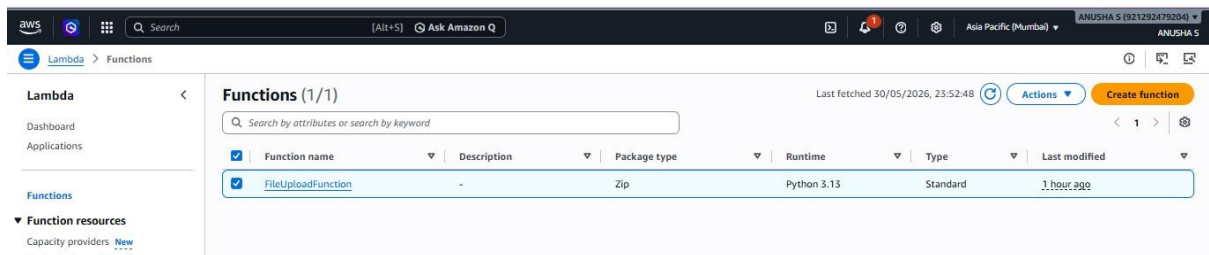


Step 5: Create Lambda Function

Create Lambda function for upload processing.

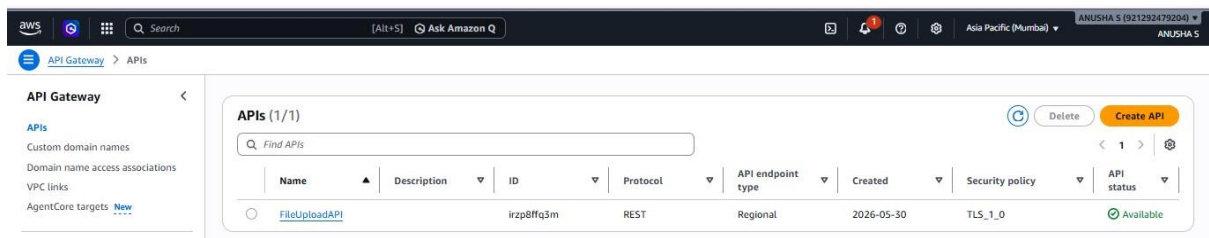
Responsibilities:

- Decode file
- Upload to S3
- Save metadata



Step 6: Configure API Gateway

Create REST API.



Configuration:

API Name: FileUploadAPI

Resource: /upload

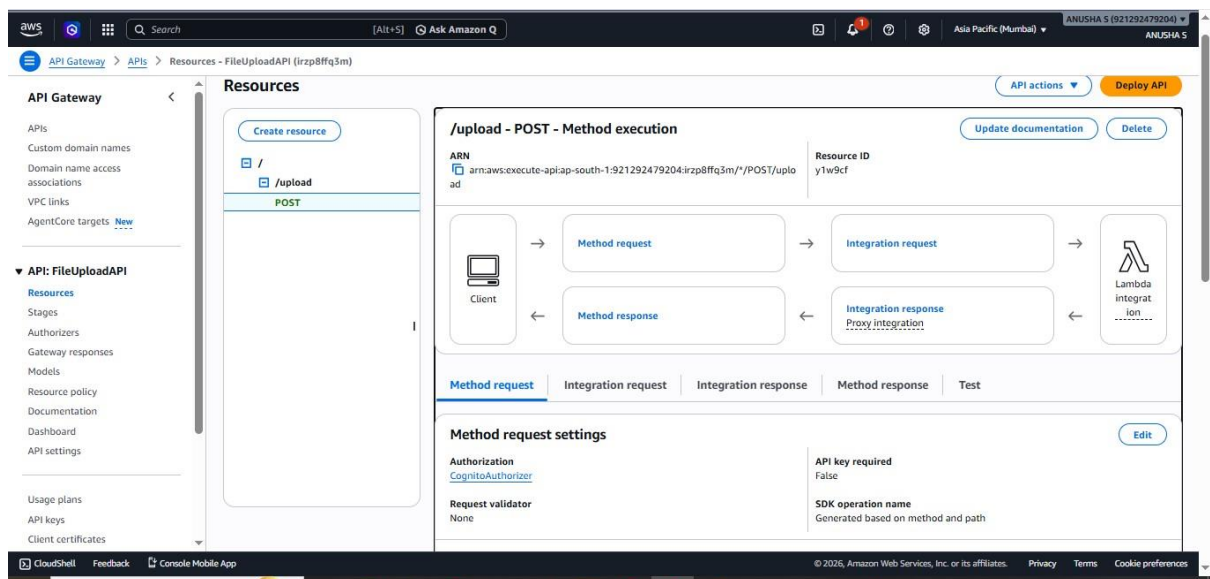
Method: POST

Step 7: Integrate API Gateway with Lambda

Connect API Gateway POST method to Lambda function.

Purpose:

- Process upload requests
- Execute backend logic

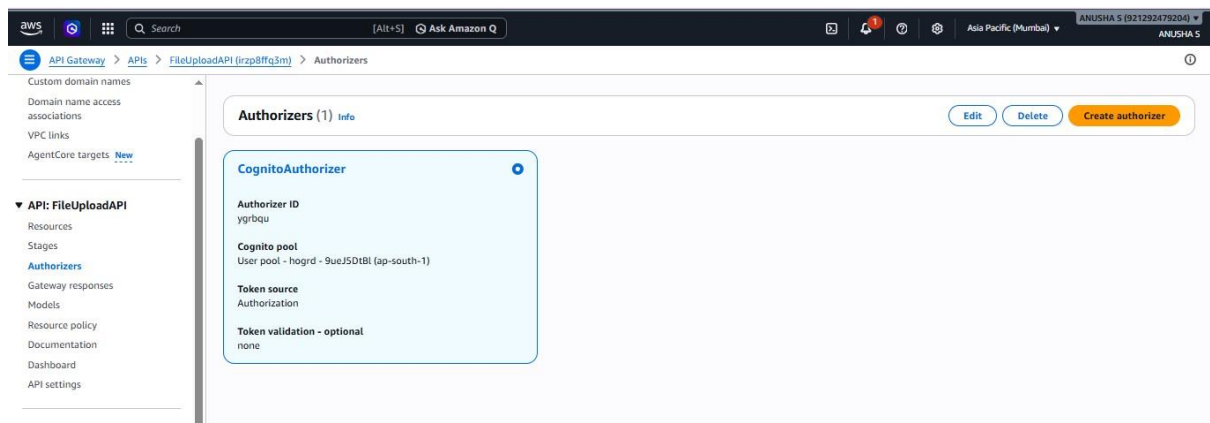


Step 8: Configure Cognito Authorization

Create Cognito Authorizer.

Purpose:

- Restrict access to authenticated users
- Validate JWT tokens



Step 9: Create Frontend Application

Develop frontend using: HTML, CSS, JavaScript

Features:

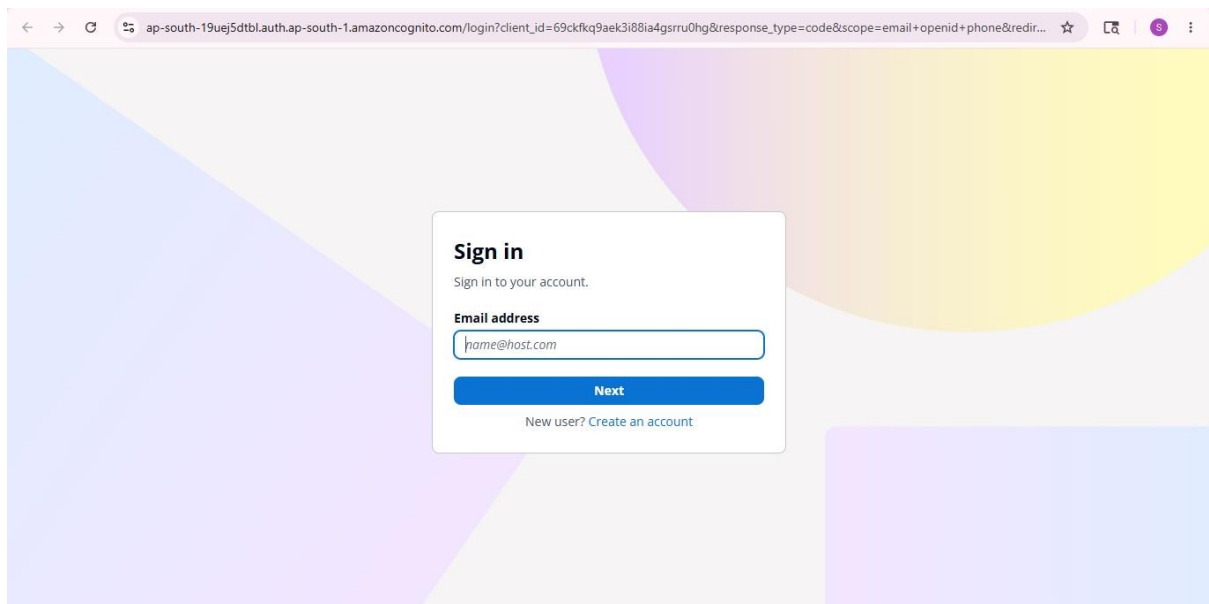
- User login
- File upload
- API integration

Step 10: Host Frontend

Host application using Amazon S3 Static Website Hosting.

Purpose:

- Public access to frontend
- Low-cost hosting solution



Troubleshooting

Issue 1

Lambda Function Not Executing

Method:

Check CloudWatch Logs.

Solution:

Verify IAM permissions and Lambda code.

Issue 2

File Not Uploaded to S3

Method:

Check S3 bucket name and Lambda logs.

Solution:

Verify bucket permissions and configuration.

Issue 3

Metadata Not Stored in DynamoDB

Method:

Check DynamoDB table configuration.

Solution:

Verify table name and IAM permissions.

Issue 4

User Authentication Failure

Method:

Check Cognito User Pool configuration.

Solution:

Verify Client ID, User Pool ID, and callback URLs.

Issue 5

API Gateway Returns Unauthorized

Method:

Verify Cognito Authorizer configuration.

Solution:

Ensure valid JWT token is included in request headers.

Conclusion

This project successfully demonstrates the implementation of a fully serverless web application using AWS cloud services.

By integrating Amazon Cognito, API Gateway, Lambda, S3, and DynamoDB, the system provides:

- Secure user authentication
- Scalable serverless processing
- Reliable file storage
- Efficient metadata management
- High availability
- Cost optimization

The project follows modern cloud-native architecture principles and serves as a practical implementation of serverless application development on AWS. It provides valuable hands-on experience in authentication, API integration, storage services, and event-driven computing, making it an excellent project for Cloud Engineer and DevOps Engineer portfolios.